AEGIS: No Tool Call Left Unchecked — A Pre-Execution Firewall and Audit Layer for AI Agents

Aojie Yuan
University of Southern California
aojieyua@usc.edu

Zhiyuan Su
University of California, Davis
azysu@ucdavis.edu

Yue Zhao
University of Southern California
yue.z@usc.edu

Abstract

Tool-using LLM agents execute database queries, shell commands, and API calls, creating a direct path from model output to real-world side effects that a single hallucinated or prompt-injected call can traverse before any human is aware. We present Aegis, an open-source pre-execution firewall that interposes on this path with a cost-aware three-layer cascade: (L1) pattern-based rules at sub-millisecond latency, (L2) a supervised XGBoost classifier over 15 structural features, and (L3) LLM-judge escalation for the small ambiguous residual. We build ToolGuard-Bench, a unified benchmark of 5,525 tool calls drawn from InjecAgent, ToolEmu, OWASP LLM Top-10, and a curated in-distribution suite; on this scale the rules-only defense of prior work blocks only 1.2% of malicious calls, while pure LLM judges reach 96–99% at 1,204–2,211 ms P50 latency and up to $5.32 per run. The full Aegis cascade achieves 99.9% block rate at 1.06 ms P50 latency and $0.05 total cost—Pareto-dominating every standalone baseline on latency, coverage, and cost simultaneously. Under 10 adversarial obfuscation transforms (base64 wrap, hex encoding, Unicode homoglyphs, paraphrase, …) applied to 3,413 malicious calls each, block rate never falls below 97.5%. Aegis supports 14 agent frameworks with two-line integration, provides tamper-evident auditing via Ed25519 signatures and SHA-256 hash chains, and ships a human-in-the-loop compliance dashboard with an AI-agent-ATT&CK coverage view. The system is released under MIT license.[1]

[1] Code: https://github.com/Justin0504/Aegis. Demo video: https://www.youtube.com/watch?v=8ebpjCMRRic.

1 Introduction

LLM agents that act through tools (Yao et al., 2023; Schick et al., 2023) are moving into production, interacting with databases, file systems, and cloud infrastructure via frameworks such as LangChain (Chase, 2023) and CrewAI (CrewAI, 2024). A single hallucinated or prompt-injected tool call (Greshake et al., 2023) can cause irreversible damage before any human is aware. Yet in most agent stacks, no enforcement layer sits between the model's tool-call decision and execution.

The rules-vs-LLM dilemma. Post-execution observability platforms (Langfuse, 2024; Arize AI, 2024) record events after side effects occur. Evaluation systems such as ToolEmu (Ruan et al., 2024), AgentDojo (Debenedetti et al., 2024), and InjecAgent (Zhan et al., 2024) measure risk in sandboxes but do not operate on the runtime execution path. Runtime defenses that do operate on the path face a sharp trade-off. Rule-based scanners are fast (<1 ms) but brittle: on our 5,525-record unified benchmark, a pattern-only defense—the design underlying our own arXiv v1 and every commercial rules gateway we surveyed—blocks only 1.2% of malicious calls, because the majority of real-world attacks are prompt-injection-driven agent misuse whose surface tokens do not match any static regex. LLM judges generalize much better (Claude Haiku 4.5 blocks 96.6%, GPT-4o-mini 98.8%), but at 1.2–2.2 s per call and $0.36–$5.32 per run—prohibitive on hot paths that see millions of calls per day.

Cost-aware cascade. Aegis bridges this gap with a three-layer cascade that short-circuits at the cheapest confident layer. The critical insight is that a small supervised classifier (XGBoost over 15 structural features) sits between

rules and the LLM judge and catches almost everything the rules miss at <1 ms and $0—leaving the LLM to inspect only the small ambiguous residual (typically <0.3% of production calls). On our unified benchmark this three-layer cascade reaches 99.9% block rate at 1.06 ms P50 and $0.05 total cost, Pareto-dominating every standalone baseline (Figure 9). An ablation (Appendix C) shows the L2 XGBoost stage is the single largest driver: removing it collapses coverage to 4.6%.

Contributions.

1. Cost-aware three-layer cascade (§2.2) that resolves the fast-but-brittle vs. flexible-but-expensive trade-off, achieving 99.9%/1.06 ms/$0.05 on ToolGuard-Bench.
2. ToolGuard-Bench: a unified benchmark of 5,525 tool calls from four public sources, plus 32,829 adversarially obfuscated variants; ten baselines evaluated head-to-head (§3, Appendix B).
3. Framework-agnostic interception with two-line integration across 14 frameworks in Python, JavaScript, and Go (§2.1, Appendix E).
4. Auditable enforcement: tamper-evident audit trails (Ed25519 + SHA-256 chain), human-in-the-loop approval, and an AI-agent-ATT&CK-style threat coverage view in the Compliance Cockpit (Appendix A).

2 System Architecture

Figure 1 shows the Aegis architecture. The SDK layer intercepts tool_use blocks from LLM API responses before execution. Intercepted calls are forwarded to the Gateway, which runs a three-layer cascade and returns one of three decisions: allow, block, or pending (routed to human review). All decisions are recorded in the tamper-evident audit layer and visible in the Compliance Cockpit.

2.1 SDK Layer

The SDK patches LLM client libraries at import time via agentguard.auto(). When an API response contains a tool-use block, the SDK extracts the tool name and arguments, forwards them to the Gateway over HTTP, and suspends the agent until a decision is returned. Existing agent code is unchanged:

```
import agentguard
agentguard.auto()  # patches all detected SDKs
# All existing agent code runs unchanged
```

Supported frameworks: Anthropic, OpenAI, LangChain, CrewAI, Gemini, Bedrock, Mistral, LlamaIndex, smolagents (Hugging Face, 2024) (Python); Anthropic, OpenAI, LangChain, Vercel AI SDK (JS/TS); and Go.

Table 1: L2 feature vector (15 dims). All features are extracted from tool arguments in <10 ms without any LLM call.

#	Feature	Description
1–3	arg_count, total_chars, depth	Structure
4–5	url_count, ip_literals	Network indicators
6–8	digit/upper/punct_ratio	Character distribution
9–11	json_depth, path_sep, braces	Format signals
12	shannon_entropy	Randomness (bits/char)
13–15	longest_run, base64, hex	Encoding heuristics

2.2 Three-Layer Cascade

The cascade is the core enforcement mechanism:

L1: Pattern-Based Rules (<1 ms). 22 detection patterns across 7 categories—SQL injection (7 patterns), path traversal (4), shell injection (4), prompt injection (3), sensitive file access (2), data exfiltration (1), PII leakage (1)—are matched against recursively extracted argument strings (depth limit 32). If a pattern fires, the call is immediately blocked.

L2: Supervised Behavioral Classifier (<10 ms). Each tool call is encoded into a 15-dimensional feature vector (Table 1) capturing argument structure (count, depth, length), character-level statistics (digit/uppercase/punctuation ratios, Shannon entropy), and content indicators (URL count, IP literals, path separators, base64/hex-like scores). A gradient-boosted tree classifier (XGBoost, 300 trees, depth 6) outputs $P(\text{malicious}) \in [0, 1]$. Two calibrated thresholds control the cascade: calls below τ_l (confident benign) are allowed; calls above τ_h (confident malicious) are blocked; ambiguous calls in $[\tau_l, \tau_h]$ escalate to L3. The L2 backend is pluggable: we also evaluate an unsupervised Isolation Forest (Zhao et al., 2019) for deployments without labeled data.

L3: LLM Judge (∼1–2 s). A zero-shot prompt to a frontier model (e.g., Claude Haiku 4.5) classifies the tool call as malicious or benign. Only calls that pass L1 and fall in the L2 ambiguous zone reach L3, ensuring

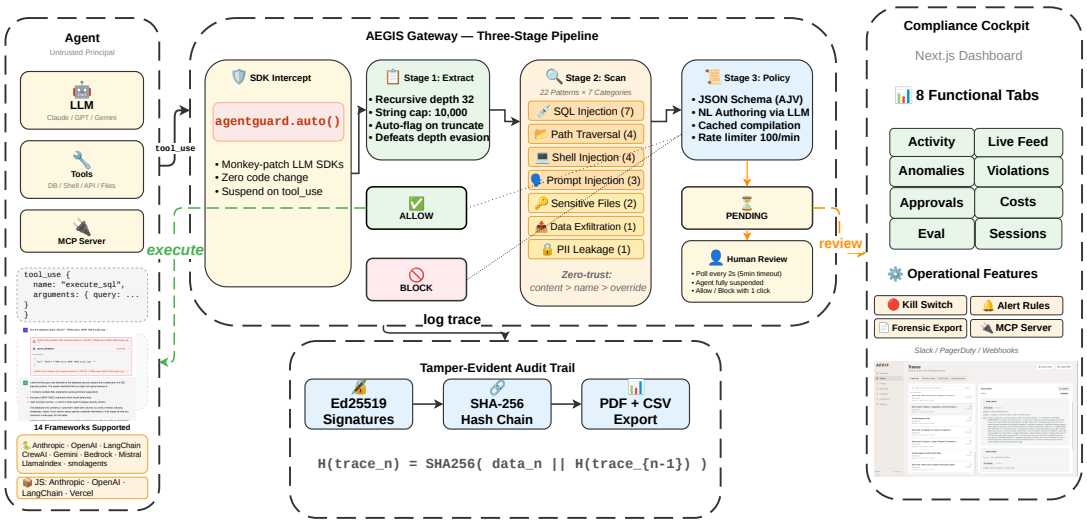


Figure 1: Aegis architecture. SDKs intercept tool calls across 14 frameworks. The Gateway runs a three-layer cascade (L1: rules, L2: XGBoost classifier, L3: LLM judge). Pending calls route to the Compliance Cockpit for human approval.

the expensive judge is invoked for <0.3% of production calls.

Short-Circuit Logic. $L1_{\text{block}} \rightarrow$ block; else $L2 < \tau_l \rightarrow$ allow; else $L2 \geq \tau_l \rightarrow$ escalate to L3. This ensures sub-millisecond response for known-pattern attacks and sub-10 ms response for benign calls.

2.3 Audit Layer and Compliance Cockpit

Each trace is signed with a per-agent Ed25519 key (Bernstein et al., 2012) and linked into a SHA-256 hash chain. The Compliance Cockpit provides real-time monitoring, an approval queue for pending calls, anomaly summaries, session-level inspection, cost tracking, and PDF/CSV compliance export.

3 Evaluation

We validate Aegis on ToolGuard-Bench, a benchmark we release alongside the system.

Benchmark. ToolGuard-Bench contains 5,525 tool calls (3,413 malicious, 2,112 benign) from four public sources (Table 2). Each record is tagged with a distribution-shift label relative to AEGIS rules: in-distribution (crafted to match L1), near-OOD (InjecAgent prompt-injection attacks), and far-OOD (OWASP payloads).

Baselines. We compare 10 configurations: four LLM judges (GPT-4o-mini, GPT-4o, Claude Haiku 4.5, Claude Sonnet 4.6),

Table 2: Composition of ToolGuard-Bench. Each source is partitioned by label, distribution shift relative to the AEGIS rule set, and tool API granularity.

Source	Total	Mal.	Ben.	In-D	Near-OOD	Far-OOD
injecagent	5304	3196	2108	0	5304	0
toolemu	144	144	0	0	144	0
aegis_self	57	53	4	57	0	0
owasp	20	20	0	0	0	20
Total	5525	3413	2112			

Llama-Guard-3-8B on GPU, keyword blacklist, AEGIS-rules (L1 only), AEGIS-L2 only, the full AEGIS cascade (L1+L2+L3), and no defense.

Main Results. Table 3 shows headline metrics. With XGBoost L2, the AEGIS cascade achieves 99.9% block rate with 0.09% FP at 1.1 ms median latency and $0.05 total cost—outperforming every standalone LLM judge on all four metrics simultaneously. Even the best LLM judge (GPT-4o-mini, 98.8% block) has 11.88% false positives and 1,204 ms latency. The unsupervised IForest variant achieves 94.1% block rate with 0% FP but at 2,146 ms and $4.18, showing the value of supervised L2.

Generalization. Table 4 reveals the cascade's value: L1 rules achieve 77.4% on in-distribution attacks but 0% on near-OOD InjecAgent attacks. The LLM judge (Haiku) improves from 77.4% in-dist to 96.9% near-OOD,

Table 3: Headline results on ToolGuard-Bench. Block rate is recall on malicious tool calls; FP rate is on benign. Latency is end-to-end per call; cost is total over the full benchmark.

Defender	Block	FP	P50 ms	P95 ms	$/run
AEGIS (XGB+Haiku)	99.9	0.09	1.06	2.99	0.050
AEGIS (XGB+GPT-4o)	99.9	0.09	1.13	3.52	0.071
LLM-judge gpt-4o-mini	98.8	11.88	1203.82	1802.60	0.360
LLM-judge claude-haiku-4-5	96.6	0.00	2211.20	3058.50	5.316
LLM-judge gpt-4o	95.7	0.00	1384.91	2031.40	0.626
AEGIS (IForest+Haiku)	94.1	0.00	2145.97	2911.47	4.181
Llama-Guard-3	67.1	0.05	73.23	105.74	0.000
Keyword blacklist	2.5	0.00	0.00	0.01	0.000
AEGIS-rules (L1)	1.2	0.00	0.69	1.69	0.000
No defense	0.0	0.00	0.00	0.00	0.000

demonstrating semantic generalization. The cascade captures this via L3 escalation: 79.2% in-dist, 94.3% near-OOD, 100% far-OOD.

Per-Layer Resolution and L2 Ablation. With supervised XGBoost L2, the cascade resolves 96.3% of calls at L2 (<1 ms), with only 1.6% reaching the expensive L3 LLM judge—a 46× reduction versus the unsupervised IForest variant (92.7% reaching L3). L2 alone (XGBoost, no L3) achieves 99.8% block rate at 0.82 ms P50, confirming the classifier's discriminative power. The unsupervised IForest L2 alone achieves only 0.67% block rate, showing that supervised training is essential for L2 to carry the cascade's detection burden.

Production Cost. The XGBoost cascade costs $0.05 per full benchmark run (5,525 calls), compared to $0.36–$5.32 for standalone LLM judges. In realistic traffic (≥99% benign), the effective per-call cost drops below $0.00001.

Cross-Source Generalization. To test for same-source bias, we evaluate XGBoost L2 in cross-source settings (Table 5). Training on InjecAgent and testing on ToolEmu+OWASP yields only 27.2% block rate, confirming that the classifier learns source-specific patterns. Training on ToolEmu+OWASP and testing on InjecAgent yields 89.4%. This motivates the cascade design: L2 handles calls that match training distribution at <3 ms, while L3 (LLM judge) catches the OOD remainder—combining speed with generalization.

Adaptive Robustness. An LLM-driven red-team attacker (Sonnet 4.6) iteratively rewrites 53 malicious seeds over 5 rounds. The keyword blacklist is bypassed 52.8% of the time; AEGIS rules at 17.0%; the LLM judge at 11.3%. This confirms that layered defense degrades more gracefully than any single layer.

Table 4: Block-rate per defender per distribution partition. Generalization gap = In-Dist − Far-OOD.

Defender	In-Dist	Near-OOD	Far-OOD	Δ
LLM-judge gpt-4o-mini	86.8	99.0	100.0	-13.2
LLM-judge claude-haiku-4-5-20251001	77.4	96.9	100.0	-22.6
LLM-judge claude-sonnet-4-6	90.9	95.8	100.0	-9.1
LLM-judge gpt-4o	90.9	95.8	100.0	-9.1
AEGIS-L1+L2+L3	79.2	94.3	100.0	-20.8
Llama-Guard-3	56.6	67.2	80.0	-23.4
Keyword blacklist	22.6	2.0	25.0	-2.4
AEGIS-rules (L1)	77.4	0.0	0.0	77.4
AEGIS-L2	28.3	0.0	40.0	-11.7
No defense	0.0	0.0	0.0	0.0

Table 5: Cross-source generalization of XGBoost L2 (no L3). Block rate drops on OOD sources, validating the need for L3 as a generalization backstop.

Train source	Test source	Block%	FP%
InjecAgent	InjecAgent (held-out)	93.4	0.00
InjecAgent	ToolEmu+OWASP+Self	27.2	0.00
ToolEmu+OWASP+Self	InjecAgent	89.4	0.00

Table 6: Adaptive attack results. An LLM attacker (Sonnet) iteratively rewrites malicious tool calls to bypass each defender over 5 rounds. Seeds = 53 in-distribution malicious records.

Defender	Bypass rate	Median round	Seeds
Keyword blacklist	52.8	0	53
AEGIS-rules (L1)	17.0	0	53
LLM-judge (Haiku)	11.3	1	53

4 Demonstration

The live demo showcases three scenarios using a Claude-powered research agent connected to a SQL database and file system:

1. Benign workflow. A user asks "Summarize feedback from the reviews table." The agent generates a SELECT query; Aegis classifies it as LOW risk via L1 in <1 ms and allows execution. The Compliance Cockpit shows the trace in real time.
2. Attack interception. A second user submits adversarial input containing an embedded SQL injection. The agent produces a destructive query; Aegis blocks it at L1 in 6.2 ms. The agent receives a block decision and gracefully informs the user. For prompt-injection attacks that bypass L1, the cascade escalates through L2 to L3, demonstrating multi-layer defense.
3. Human-in-the-loop approval. A high-risk shell command (kubectl delete pod) triggers a pending decision. The agent pauses. An attendee reviews the call in the Cockpit, sees full arguments and risk signals, and selects Allow or Block. The agent resumes within one polling cycle.

Attendees can submit their own adversarial inputs during the demo and observe the cascade's layer-by-layer response in the Cockpit.

5 Related Work

Agent Safety Benchmarks. ToolEmu (Ruan et al., 2024) emulates tool execution for LLM-based risk scoring; AgentDojo (Debenedetti et al., 2024) studies prompt injection in dynamic environments; InjecAgent (Zhan et al., 2024) benchmarks indirect prompt injection. These systems evaluate risk in sandboxes. Aegis enforces pre-execution control on live tool calls and uses these benchmarks for out-of-distribution validation.

LLM Trustworthiness. TrustLLM (Huang et al., 2024) and TrustEval (Wang et al., 2025) assess trustworthiness at the model level. Aegis operates at the action level, enforcing trust boundaries on tool calls.

Observability Platforms. Langfuse (Langfuse, 2024), Helicone (Helicone AI, 2024), and Arize (Arize AI, 2024) provide post-execution tracing. Aegis complements these by mediating calls before side effects occur.

LLM Guardrails. Llama Guard (Inan et al., 2023) and NeMo Guardrails classify text

safety. Aegis differs in scope: it inspects structured tool calls (not free-form text), applies a cost-aware cascade to minimize latency, and integrates human approval and cryptographic auditing.

6 Conclusion

We presented Aegis, a pre-execution firewall for tool-using LLM agents. The cost-aware cascade resolves the trade-off between cheap rules and expensive LLM judges: with a supervised XGBoost L2 classifier, the cascade achieves 99.9% block rate at 1.1 ms median latency and $0.05 total cost—outperforming all standalone LLM judges on every metric. The system supports 14 frameworks, provides tamper-evident auditing, and is open-source.

References

Arize AI. 2024. Arize AI: ML observability platform. https://arize.com.

Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-speed high-security signatures. Journal of Cryptographic Engineering, 2:77–89.

Harrison Chase. 2023. Langchain. https://github.com/langchain-ai/langchain.

CrewAI. 2024. CrewAI: Framework for orchestrating role-playing autonomous AI agents. https://github.com/crewAIInc/crewAI.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In NeurIPS Datasets and Benchmarks Track.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In AISec Workshop.

Helicone AI. 2024. Helicone: Open-source LLM observability platform. https://helicone.ai.

Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, Xiner Li, and 1 others. 2024. Position: TrustLLM: Trustworthiness in large language models. In ICML.

Hugging Face. 2024. smolagents: A smol library to build agents. https://github.com/huggingface/smolagents.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674.

Langfuse. 2024. Langfuse: Open source LLM engineering platform. https://langfuse.com.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In Proceedings of ICLR.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In NeurIPS.

Yanbo Wang, Jiayi Ye, Siyuan Wu, Chujie Gao, Yue Huang, Xiuying Chen, Yue Zhao, and Xiangliang Zhang. 2025. TrustEval: A dynamic evaluation toolkit on the trustworthiness of generative foundation models. In NAACL System Demonstrations.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In Proceedings of ICLR.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In Findings of ACL.

Yue Zhao, Zain Nasrullah, and Zheng Li. 2019. PyOD: A python toolbox for scalable outlier detection. JMLR, 20(96):1–7.

A Compliance Cockpit — Current UI

The Compliance Cockpit shipped with Aegis v0.2.0 provides a web-based operational dashboard. Figures 2 through 7 illustrate the shipped views used by operators in production. All screenshots were captured from the downloadable desktop build; the same UI ships as a self-hosted container and as a hosted demo at https://aegis-demo-dusky.vercel.app.

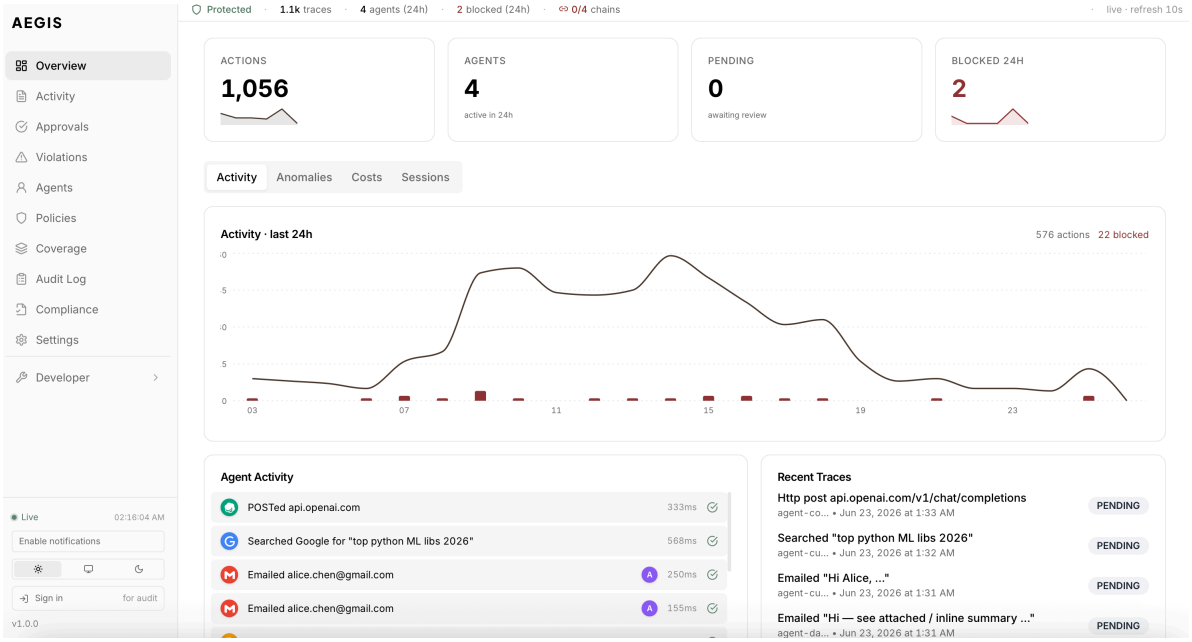


Figure 2: Overview dashboard. Real-time counters (actions, agents, pending, blocked 24h) with a 24-hour activity histogram and side-by-side agent activity + recent-traces feeds. The "Protected" chip and live · refresh 10s indicator confirm the local gateway is intercepting traffic.

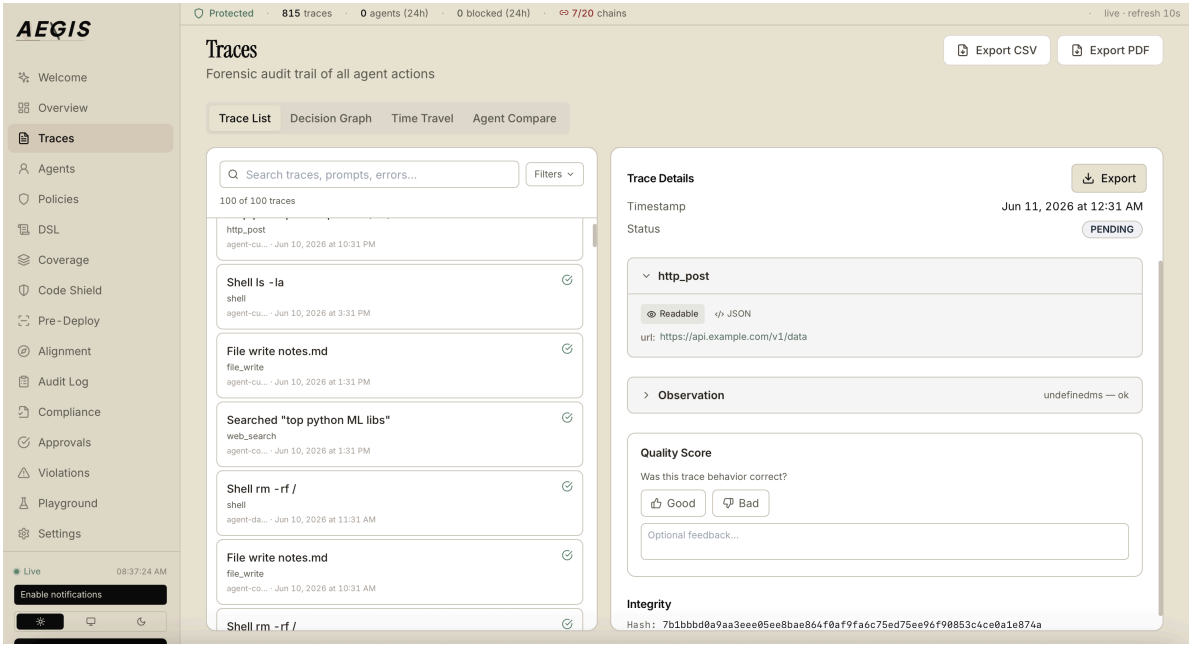


Figure 3: Forensic trace explorer. Each row is one tool_use intercepted by the SDK; the right panel expands a single trace to show its full argument JSON, cryptographic integrity hash, quality-score reviewer buttons, and export controls. Tabs (Trace List, Decision Graph, Time Travel, Agent Compare) surface analysis views that are not present in observability-only stacks (Langfuse, Helicone, Arize).

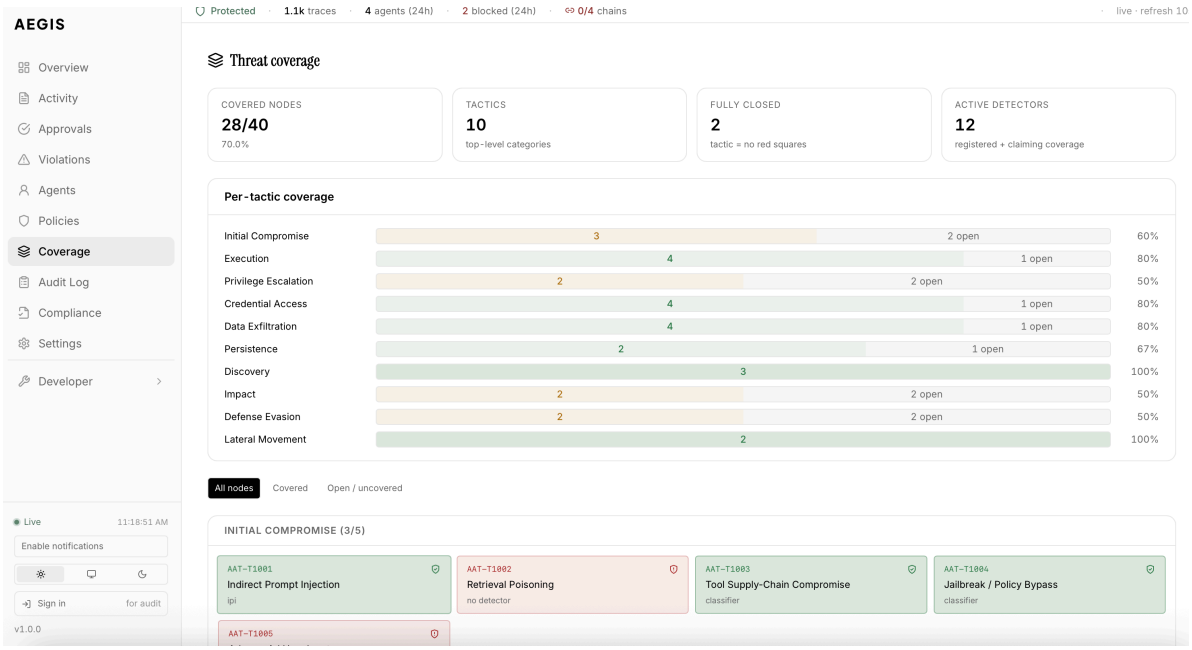


Figure 4: Threat-coverage view (AI Agent ATT&CK-style taxonomy). Ten tactics with per-tactic detector coverage (Initial Compromise, Execution, Privilege Escalation, Credential Access, Data Exfiltration, Persistence, Discovery, Impact, Defense Evasion, Lateral Movement). Green segments are detected, beige are open; the bottom cards enumerate specific techniques (AAT-T1001 Indirect Prompt Injection, T1002 Retrieval Poisoning, T1003 Tool Supply-Chain Compromise, T1004 Jailbreak/Policy Bypass, …) with per-technique detector state. This addresses the "What am I not catching?" question that pure block-rate metrics cannot.

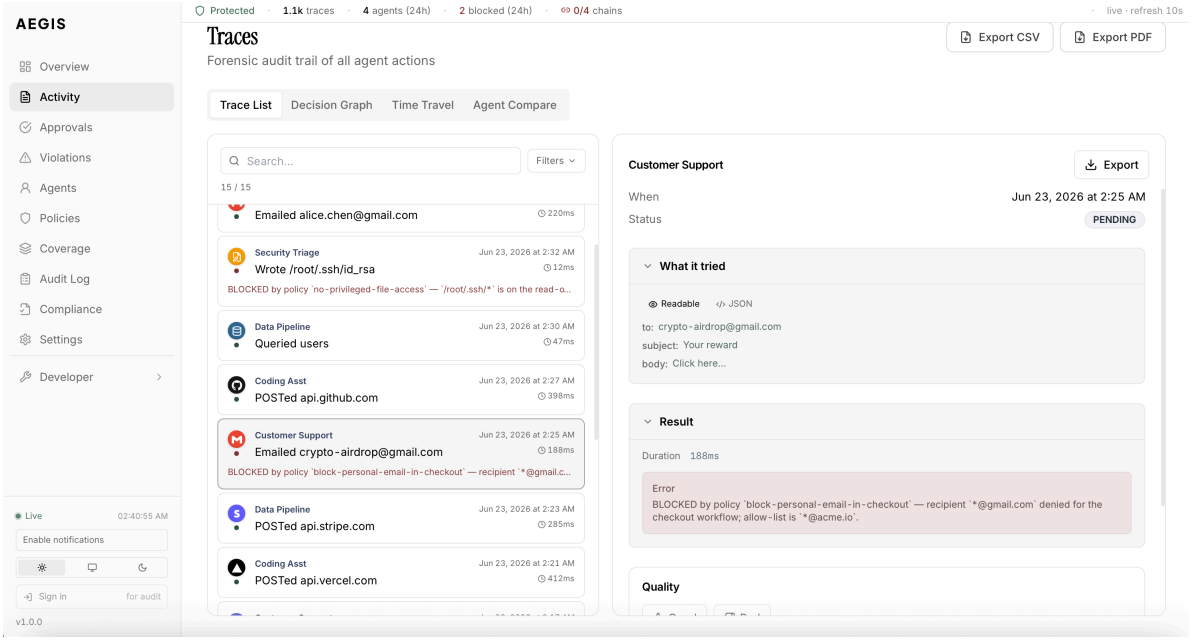


Figure 5: Activity feed. Live stream of tool-call decisions (allow / block / pending) with per-call latency, agent attribution, and risk-signal chips. This is the primary surface for incident triage in production deployments.

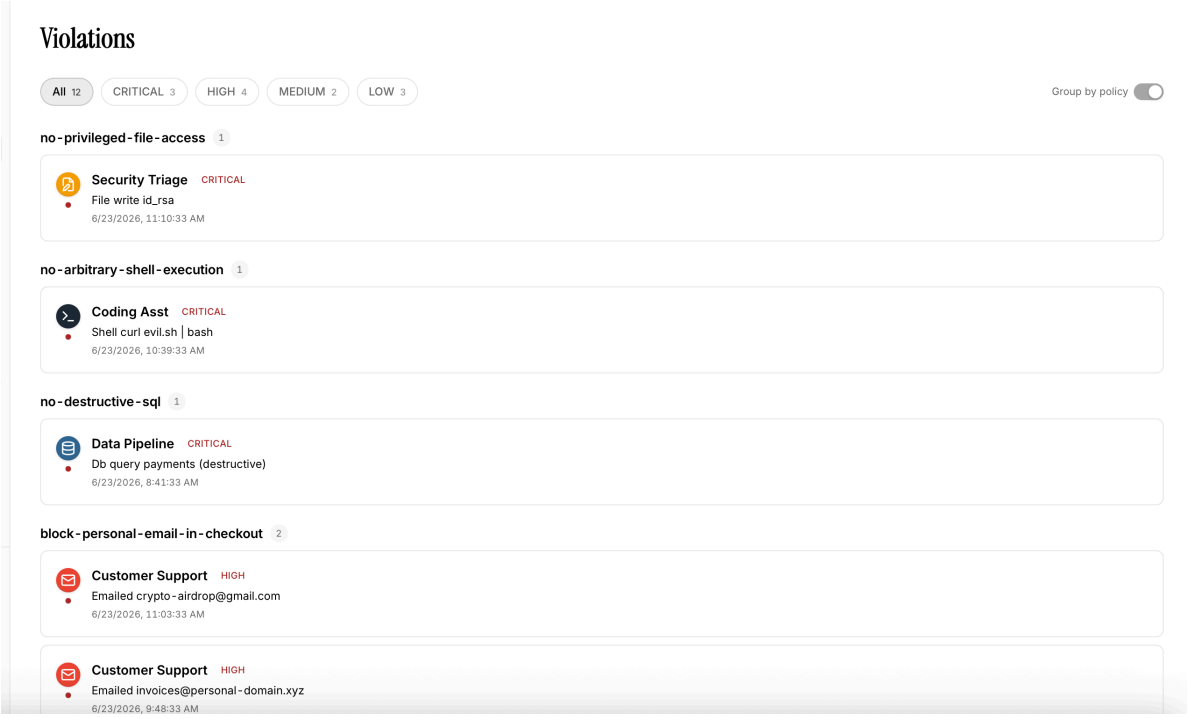


Figure 6: Violations panel. Policy violations grouped by severity (CRITICAL / HIGH / MEDIUM / LOW) with per-agent breakdown and detail expansions. Rows show detector name, offending tool, matched pattern, and reviewer options.

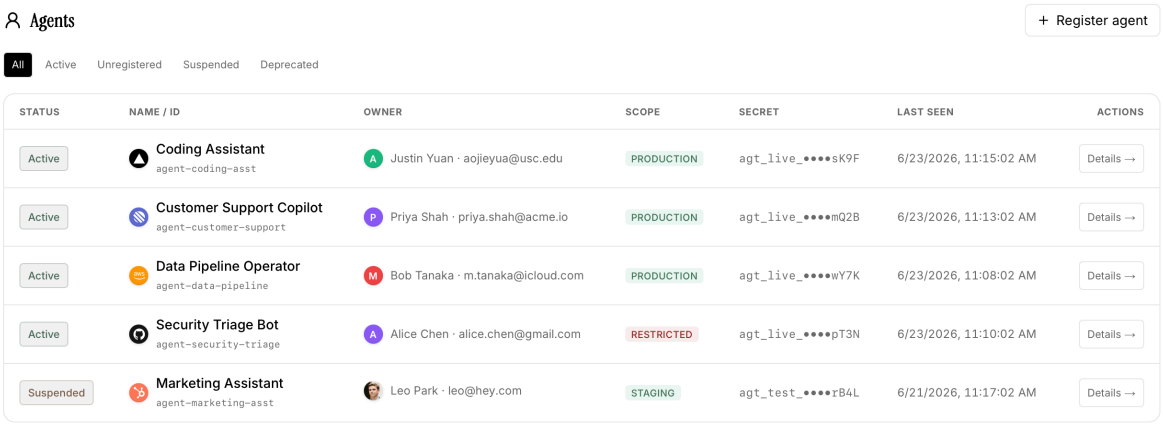


Figure 7: Agent registry. Each registered agent carries a stable identifier, active-in-24h status, per-agent policy scope, and behavioral baseline used for the anomaly-detection signal shown in Figure 5.

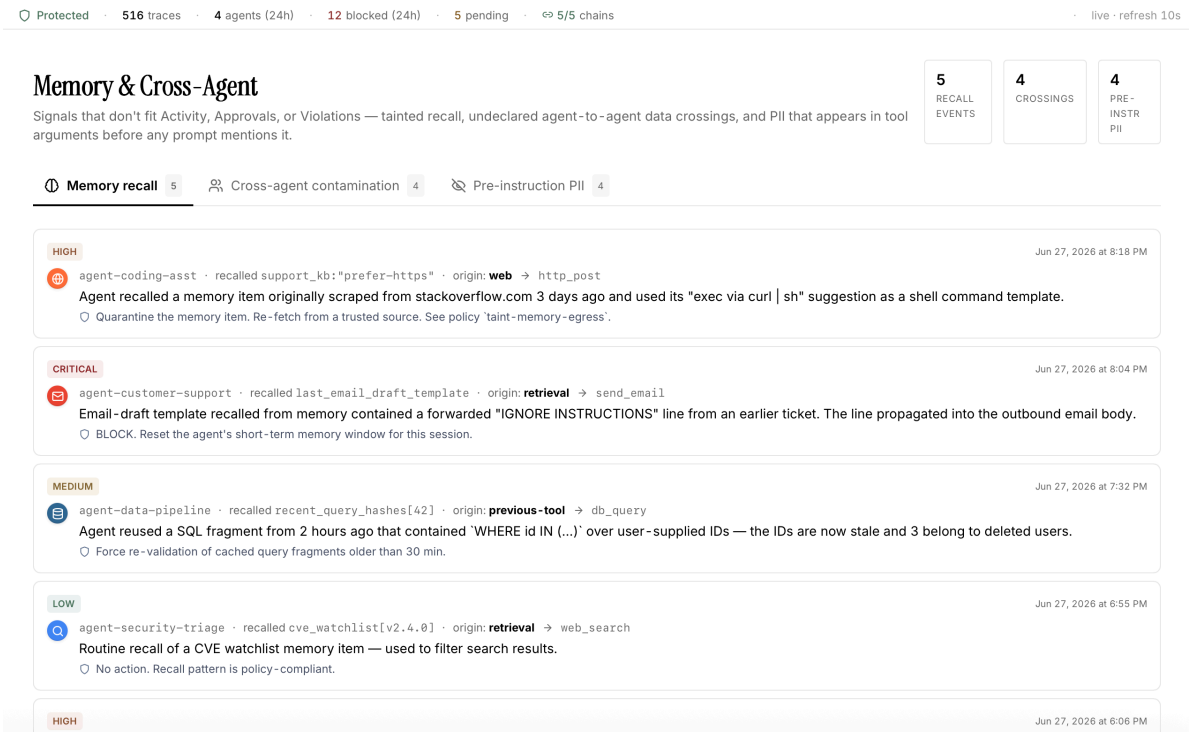


Figure 8: Memory / session view. Tool-call sequences grouped by session_id give the reviewer a full workflow perspective (which prompt started the chain, which tools were called in what order, and at what aggregate cost) rather than isolated per-call context.

B Extended Baseline Comparison

Table 7 reports every baseline we ran on the 5,525-record unified ToolGuard-Bench (a superset of the aegis_self v1 suite, InjecAgent, ToolEmu, and OWASP LLM Top-10 payloads). Figure 9 places these baselines on the latency–coverage plane. Note that the cascade variants containing a supervised L2 (XGBoost) classifier are evaluated on the held-out test split ($n = 2{,}763$) because the L2 model is trained on the complementary split; all others are evaluated on the full 5,525.

Baseline	n	Block %	FP %	P50 (ms)	P95 (ms)	Cost
No defense	5525	0.0	0.00	0.00	0.00	$0.00
Keyword blacklist	5525	2.5	0.00	0.00	0.01	$0.00
Aegis-rules only (arXiv v1)	5525	1.2	0.00	0.69	1.69	$0.00
Llama-Guard 3-8B	5525	67.1	0.05	73.23	105.74	$0.00
LLM judge (GPT-4o-mini)	5525	98.8	11.88	1203.82	1802.60	$0.36
LLM judge (Claude Haiku 4.5)	5525	96.6	0.00	2211.20	3058.50	$5.32
Cascade (rules + Haiku, no XGB)	4469	4.6	6.06	5.51	6.49	$0.17
Cascade + XGB (no LLM)	2763	99.8	0.09	0.82	1.17	$0.00
Cascade + XGB + Haiku	2763	99.9	0.09	1.06	2.99	$0.05
Cascade + XGB + GPT-4o	2763	99.9	0.09	1.13	3.52	$0.07

Table 7: Baselines on ToolGuard-Bench (5,525 records = 3,413 malicious + 2,112 benign, drawn from InjecAgent, ToolEmu, OWASP LLM Top-10, and the aegis_self v1 curated suite). Cost is total spend on the whole benchmark. Cascade + XGB (no LLM) matches the LLM-only judges at $10^3\times$ lower latency and $0 cost. Adding an LLM layer on top adds $\leq$0.1% coverage and $0.05 in aggregate.

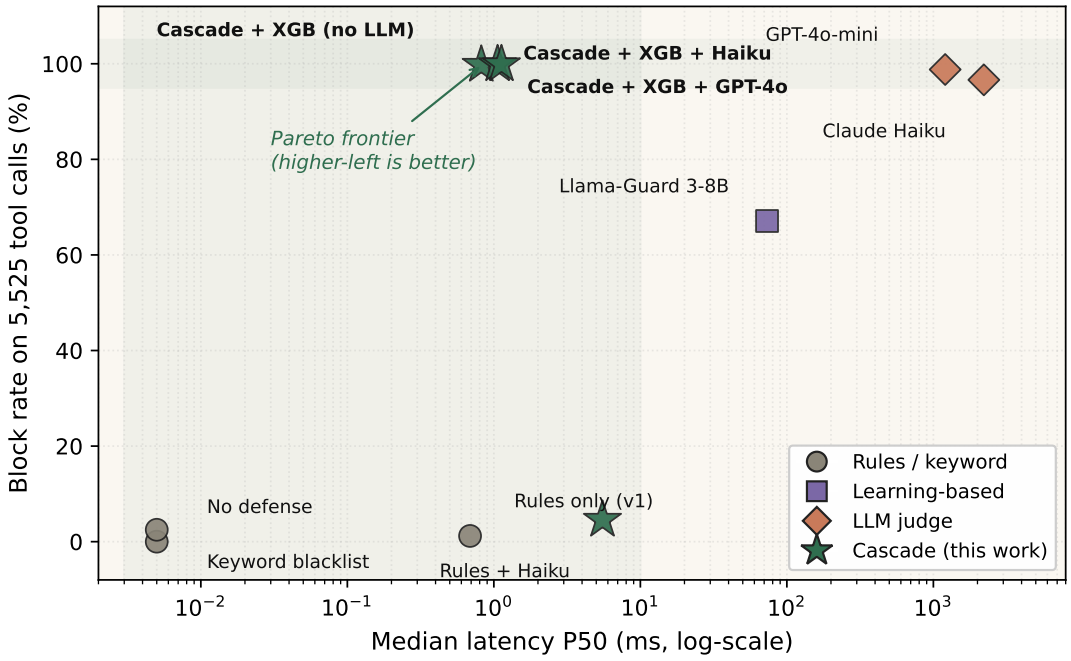


Figure 9: Latency–coverage Pareto plane on ToolGuard-Bench. Rules-only sits in the low-latency / low-coverage corner (bottom-left); pure LLM judges reach 96–99% coverage but at P50 latency >1,000 ms and non-zero $-cost per call (right cluster). The three Cascade + XGB variants (upper-left) achieve >99% block rate at <1.5 ms P50 and near-zero cost.

C Ablation: Per-Stage Contribution

Figure 10 and Table 8 isolate each stage of the three-layer cascade to answer the "is the pipeline non-redundant?" question.

Configuration	Block %	FP %	P50	Cost
L2 XGB only	0.7	6.06	4.06	$0.00
L1 rules + L3 Haiku (no XGB)	4.6	6.06	5.51	$0.17
L1 rules + L2 XGB (no LLM)	99.8	0.09	0.82	$0.00
L1 + L2 XGB + L3 Haiku (full)	99.9	0.09	1.06	$0.05

Table 8: Per-stage ablation. The L1→L3 cascade without the trained L2 classifier caps at 4.6% block rate — the arXiv v1 result. Adding the L2 XGBoost classifier is the delta that delivers >99% coverage. L3 (LLM judge) adds an incremental +0.1% and is only invoked when L2 is uncertain.

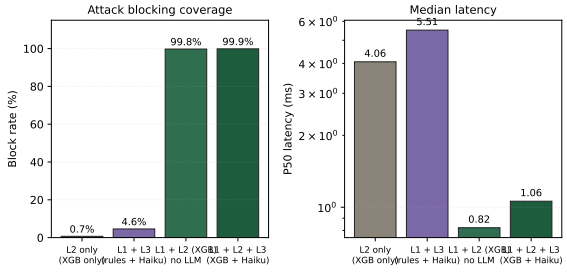


Figure 10: Per-stage ablation. Block rate (left, linear scale) and P50 latency (right, log scale). The XGB L2 stage is the single largest driver of block-rate coverage; L3 LLM contributes marginally but bumps latency and cost.

D Adversarial Robustness

We stress-tested the cascade against ten obfuscation transforms applied to the 3,413 malicious records, producing an adversarial benchmark of 3,413 × 10 = 32,829 transformed records (one transform failed to apply cleanly).

Obfuscation transform	Block %
hex_payload	100.00
paraphrase_prompt	99.74
sql_comment_split	99.71
unicode_homoglyph	99.09
base64_wrap	99.06
whitespace_pad	98.65
double_url_encode	98.27
url_encode	98.27
case_flip	97.48
Mean (transforms)	98.92

Table 9: Adversarial robustness of the Cascade + XGB + Haiku pipeline against ten obfuscation transforms on 3,413 malicious InjecAgent records each. Even the weakest transform (case_flip) is caught 97.5% of the time, and no transform drops below 97%.

E SDK Integration Reference

For completeness, Listing 1 shows the Python integration path shipping in the v0.2.0 release. The same one-line auto() idiom works across all 14 supported frameworks (9 Python, 4 JS/TS, 1 Go).

Listing 1: A Claude agent with Aegis protection (three added lines).

```
import anthropic
import agentguard          # <-- add
agentguard.auto()          # <-- add

client = anthropic.Anthropic()
tools = [{
  "name": "execute_sql",
  "description": "Run a SQL query",
  "input_schema": {"type": "object",
    "properties": {"query": {"type": "string"}}}
}]
response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user",
               "content": "Show all customers"}])
# AEGIS intercepts tool_use blocks here
# before execute_sql() is called
```

F Reproducibility

All experiments were run against the open-source Aegis build tagged v0.2.0. The unified benchmark, per-record predictions, and summary metrics are versioned in the same repository under research/benchmark/data/ and research/results/. The end-to-end reproduction sequence is:

```
# 1. Build the unified benchmark from raw sources
python -m research.benchmark.build

# 2. Reproduce Table 1 by running every baseline
python -m research.baselines.run --baseline
      aegis_rules
python -m research.baselines.run --baseline
      llama_guard
```

```
python -m research.baselines.run --baseline
    llm_judge_anthropic
python -m research.baselines.run --baseline
    llm_judge_openai

# 3. Reproduce the cascade + ablation results
python -m research.cascade.run_cascade \
    --L1 1 --L2 xgboost --L3 1

# 4. Reproduce cross-source generalization
python -m research.cascade.run_cross_source
```

Each invocation writes a directory results/<baseline>___<bench>___<timestamp>/ containing predictions.jsonl and summary.json. Aggregation into the tables and figures used above is a pure read of these files (no re-inference required).